

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	3
1 Исследовательская часть	3
1.1 Обзор предметной области	3
1.2 Общие определения	4
1.3 Фактический язык шаблона	6
1.4 Регулярное представление шаблонов	8
1.4.1 Каскадное произведение автоматов	9
1.4.2 Переход от каскада к ДКА	12
1.5 Сужение шаблона как регулярный язык	13
2 Конструкторская часть	16
2.1 Общая архитектура программного комплекса	16
2.2 Архитектура подсистемы семантического анализатора	17
2.2.1 Вспомогательные подмодули семантического анализатора	17
2.2.2 Проектирование алгоритмов	18
2.3 Пользовательский интерфейс	19
3 Технологическая часть	21
3.1 Обзор используемых технологий	21
3.2 Реализация модуля для работы с абстрактными шаблонами	21
3.3 Реализация модуля для работы с автоматами	21
3.4 Реализация алгоритма обнаружения перекрытий	21
3.5 Реализация алгоритма извлечения документации	21
3.6 Руководство пользователя	21
4 Аналитическая часть	21
ЗАКЛЮЧЕНИЕ	22
СПИСОК ЛИТЕРАТУРЫ	23

АННОТАЦИЯ

ВВЕДЕНИЕ

1 Исследовательская часть

1.1 Обзор предметной области

Язык программирования Рефал имеет довольно большую историю, начавшуюся «теоретически» еще в 1966 году, когда В.Ф. Турчин предложил Рефал как метаязык для описания других языков программирования, а «практически» в 1968 г., когда появилась первая реализация Рефала. Рефал имеет множество диалектов, но все они, по сути, являются надстройками над так называемым базисным Рефалом. Базисный рефал в некотором смысле является самым простым диалектом языка. Программы на базисном Рефале состоят из функций, каждая функция задается парами из шаблона в левой части и выражения в правой.

Единственный тип данных в Рефале — объектное выражение. Это выражение, состоящее из атомарных символов (символ, число) и скобок, при этом скобки правильно вложены. Шаблон в левой части функции — это выражение, в котором, помимо скобок и атомов, могут встречаться переменные. Переменные в шаблоне могут быть трех типов: *e*-переменные, *s*-переменные и *t*-переменные. *e*-переменные могут сопоставляться с любым объектным выражением, *s*-переменные сопоставляются с атомом, *t*-переменные сопоставляются либо с атомом, либо с выражением в скобках. Каждая переменная обладает именем, при этом, если в одном шаблоне есть несколько переменных с одним именем, то при подстановке они заменяются на разные значения.

Основной механизм работы Рефала — сопоставление с образцом и переписывание выражений. При сопоставлении с образцом шаблон в левой части функции сопоставляется с объектным выражением в правой части. Если сопоставление прошло успешно, переменным присваиваются конкретные значения, затем вычисляется выражение в правой части. Сопоставление происходит сверху вниз, если какое-то выражение не сопоставилось ни с одним предложением, это приводит к ошибке времени выполнения.

В листинге 1 функция `IsPal` проверяет, является ли переданное ей выражение, состоящее только из атомов, палиндромом.

Листинг 1 — Пример функции на базисном Рефале

```
1 IsPal {  
2   s.x e.x s.x = <IsPal e.x>;  
3   = True;  
4   s.oneChar = True;  
5   e.otherwise = False;  
6 }
```

Хотя Рефал преимущественно изучается в контексте суперкомпиляции, программы на нём можно исследовать и в контексте статического анализа. С одной стороны, базисный Рефал является чистым функциональным языком, выполнять статический анализ для которых, обычно, легче (по сравнению с императивными языками). С другой стороны, основная конструкция Рефал-программ — приписывание — является ассоциативным, что существенно осложняет задачу анализа.

1.2 Общие определения

Определение 1 (Алфавит). Алфавитом Σ будем называть конечное, если не указано иное, множество символов.

Будем обозначать алфавиты заглавными греческими буквами.

Определение 2 (Связанные со строками понятия). Строкой $w = w_1w_2 \dots w_n$ будем называть конечную последовательность символов из Σ .

Длиной строки w будем называть число n , обозначаемое как $|w|$.

Пустым словом будем называть строку длины 0, обозначаемую как ε .

i -тый символ строки w будем обозначать как $w[i]$.

Строки будем обозначать строчными латинскими буквами.

Определение 3 (Обращение строки). Для строки $w = w_1w_2 \dots w_n$ будем называть ее обращением строку $w^R = w_n \dots w_2w_1$. Для множества строк $W = \{w_1, w_2, \dots\}$ будем называть его обращением множество $M^R = \{w_1^R, w_2^R, \dots\}$.

Определение 4 (Итерация Клини). Множеством строк Σ^* будем называть множество всех конечных строк над алфавитом Σ . $\varepsilon \in \Sigma^*$. Множество $\Sigma^* \setminus \{\varepsilon\}$ обозначим Σ^+ .

Определение 5 (Сокращенная запись конкатенации). Конкатенацией строк $u = u_1u_2 \dots u_n$ и $v = v_1v_2 \dots v_m$ будем называть строку $u \cdot v = u_1u_2 \dots u_nv_1v_2 \dots v_m$.

Если $M \subset \Sigma^*$ — некоторое множество строк и $w \in \Sigma^*$, то будем обозначать как Mw множество $\{mw \mid m \in M\}$. Аналогично для двух множеств строк M_1, M_2 будем обозначать M_1M_2 множество $\{m_1m_2 \mid m_i \in M_i\}$.

Определение 6. Множество $\Sigma^* \setminus W$ будем обозначать $\overline{W}$ и называть дополнением языка W .

Множества строк будем обозначать заглавными латинскими буквами.

Определение 7 (Строковый морфизм). Строковый морфизм $\phi: \Sigma^* \rightarrow \Gamma^*$ — это функция, которая каждому слову $w \in \Sigma^*$ ставит в соответствие слово $\phi(w) \in \Gamma^*$, сохраняющую конкатенацию, т.е. $\forall w_1, w_2 \in \Sigma^* \phi(w_1w_2) = \phi(w_1)\phi(w_2)$. Строковый морфизм однозначно задается набором значений на символах алфавита Σ . Если для некоторого $\sigma \in \Sigma$ не задано явным образом отображение, будем считать, что σ отображается в себя. В таком случае будем опускать этот факт в записи областей определения и значений морфизма.

Все морфизмы в данной работе — строковые, поэтому будем опускать слово "строковый". Будем обозначать морфизмы строчными греческими буквами.

Пример 1. Пусть $\Sigma = \Gamma = \{a, b, c\}$. Морфизм, меняющий местами a и b , можно задать следующим образом: $\phi: \{a, b\} \rightarrow \{a, b\}$, при этом

$$\phi(a) \mapsto b;$$

$$\phi(b) \mapsto a.$$

В таком способе записи неявно задано $\phi(c) = c$.

Определение 8 (Множество обязательных префиксов). Для языка $W \subseteq \Sigma^*$ его множеством обязательных префиксов будем называть множество $P \subseteq \Sigma^*$ такое, что

$$\forall w \in W \exists p \in P \exists v \in \Sigma^* (w = pv).$$

Определение 9 (Обязательный инфикс). Для языка $W \subseteq \Sigma^*$ его обязательным инфиксом будем называть слово $f \in \Sigma^*$ такое, что

$$\forall w \in W \exists v, u \in \Sigma^* (w = vfu).$$

Для $n \in \mathbb{N}$ примем $\overline{1, n} = \{1, 2, \dots, n\}$.

1.3 Фактический язык шаблона

Определение 10 (Язык правильно вложенных слов). Пусть Σ_{call} , Σ_{ret} и Σ_{int} — непесекающиеся алфавиты. Язык правильно вложенных слов $\mathcal{W}$ — это наименьшее множество, которое генерируется следующими конструкторами:

$$\begin{aligned} \varepsilon &\in \mathcal{W} && \text{(пустое слово)} \\ \gamma &\in \mathcal{W} && \text{для каждого } \gamma \in \Sigma_{\text{int}} \quad \text{(внутренний символ)} \\ w_1 \cdot w_2 &\in \mathcal{W} && \text{если } w_1, w_2 \in \mathcal{W} \\ \langle_c w \rangle_r &\in \mathcal{W} && \text{если } w \in \mathcal{W}, \langle_c \in \Sigma_{\text{call}}, \rangle_r \in \Sigma_{\text{ret}} \end{aligned}$$

Выражением будем называть слова языка правильно вложенных слов $\mathcal{W}_{\text{expr}}$, порожденного алфавитами $\Sigma_{\text{call}} = \{(\cdot)\}$, $\Sigma_{\text{ret}} = \{)\}$ и $\Sigma_{\text{int}} = \Delta$, где Δ — счетное множество символов. Абстрактным шаблоном без повторных переменных будем называть слова языка правильно вложенных слов $\mathcal{W}_{\text{pat}}$, порожденного алфавитами $\Sigma_{\text{call}} = \{(\cdot)\}$, $\Sigma_{\text{ret}} = \{)\}$ и $\Sigma_{\text{int}} = \{e, s, t\}$.

Замечание 1. Формальное описание абстрактного шаблона без повторных переменных соответствует таким шаблонам языка Рефал, в которых все переменные имеют различные имена и атомарные символы не встречаются вовсе. Например, для диалекта Рефал5- λ , поддерживающего безымянные переменные, из абстрактного шаблона можно получить обычный, заменив все символы $e(s, t)$ абстрактного шаблона на символы $e \cdot _ (s \cdot _, t \cdot _)$ шаблона Рефал5- λ .

Введем также понятие шаблона с пронумерованными переменными: каждому шаблону $p \in \mathcal{W}_{\text{pat}}$ поставим в соответствие слово над алфавитом $\Delta_{\mathbb{N}} = \{(\cdot)\} \cup \{e, s, t\} \times \mathbb{N}$ такое, что все вхождения символов e, s, t в p в шаблоне с пронумерованными переменными заменяются на буквы из $\{e, s, t\} \times \mathbb{N}$, причем i -тому вхождению символа e в p соответствует буква (e, i) (обозначим e_i). Аналогично для s и t . Скобки $(,)$ остаются на месте. Для шаблона p будем обозначать соответствующий ему шаблон с пронумерованными переменными $p_{\mathbb{N}}$.

Этот формализм нужен лишь затем, чтобы формализовать понятие сопоставления с образцом.

Соответствие переменных множествам их допустимых значений было приведено в обзоре базисного Рефала, теперь же формализуем это соответствие с учетом, что рассматриваются только абстрактные шаблоны без повторных переменных. Будем говорить, что выражение w допускает сопоставление с образцом p , если существует строковый морфизм $\phi: \Delta_{\mathbb{N}}^* \rightarrow \mathcal{W}_{\text{expr}}$, действующий следующим образом:

$$\begin{aligned}\phi((s, i)) &\mapsto \gamma \in \Sigma_{\text{int}} \quad \forall i \in \mathbb{N}; \\ \phi((t, i)) &\mapsto \gamma \in \Sigma_{\text{int}} \text{ или } \phi((t, i)) \mapsto (v), \quad v \in \mathcal{W}_{\text{expr}} \quad \forall i \in \mathbb{N},\end{aligned}$$

при этом $\phi(p_{\mathbb{N}}) = w$.

Определение 11 (Язык шаблона). Языком шаблона $p \in \mathcal{W}_{\text{pat}}$ будем называть множество всех выражений $w \in \mathcal{W}_{\text{expr}}$, которые допускают сопоставление с p . Обозначать это множество будем как $\mathcal{L}(p)$.

Каждая функция Рефала задается последовательностью предложений, каждое из которых в свою очередь содержит шаблон в левой части. Для некоторой функции f , содержащей n предложений, будем обозначать i -тый по счету (считая сверху вниз) шаблон предложения как $f.p_i$. Последовательность всех шаблонов функции $\langle f.p_1, f.p_2, \dots, f.p_n \rangle$ будем обозначать как **pat** f .

Не всегда язык шаблона $\mathcal{L}(p)$ совпадает с множеством слов, которые могут сопоставиться с этим шаблоном, находящимся где-то в теле функции. Как правило, языки шаблонов в функции не являются дизъюнктными, а значит часть $\mathcal{L}(p)$ может буквально не доходить до шаблона p , обрабатываясь предложениями выше. Для $f.p_i$ будем называть его фактическим языком множество

$$\mathcal{L}(f.p_i) \cap \overline{\bigcup_{j \in \overline{1, i-1}} \mathcal{L}(f.p_j)}.$$

Будем называть фактический язык шаблона p сужением шаблона и обозначать как $\mathcal{L}_{\text{fact}}(p)$.

Важным свойством сужением является его пустота (как языка). В таком случае имеет смысл говорить о перекрытии (экранировании) шаблона и соответствующего ему предложения.

Определение 12 (Перекрытие). Предложение i функции f перекрывается (экранируется), если $\mathcal{L}_{\text{fact}}(f.p_i) = \emptyset$.

Определим два варианта задачи распознавания перекрытия:

Определение 13 (Полная задача распознавания перекрытия). Задача распознавания перекрытия в функции f с n предложениями заключается в том, чтобы для каждого шаблона $p_i \in \mathbf{pat} f$, $i \in \overline{1, n}$ найти минимальное по включению множество $I = \{i_1, i_2, \dots, i_k\} \subseteq \overline{1, i-1}$ такое, что p_i перекрывается шаблонами $f.p_{i_1}, f.p_{i_2}, \dots, f.p_{i_k}$, или показать, что такого множества не существует.

Определение 14 (Упрощенная задача распознавания перекрытия). Упрощенная задача распознавания перекрытия в функции f заключается в том, чтобы для каждого шаблона $p_i \in \mathbf{pat} f$ определить, экранируется ли он шаблонами $p_1, p_2, \dots, p_{i-1}$.

Отметим, что решение полной задачи распознавания перекрытия неоднозначно, так как для одного и того же шаблона можно найти несколько минимальных по включению множеств индексов таких, что соответствующие им шаблоны перекрывают искомый. Пример такого случая приведён в листинге 2. Предложение 5 перекрывается предложениями 3 и 4, а также предложением 1. При этом множества $\{3, 4\}$ и $\{1\}$ несравнимы по включению.

Листинг 2 — Пример неоднозначного экранирования предложений Рефал-функции

```

1 F {
2     (t._ t._) e._ = ...;           /* 1 */
3     e._ (s._ e._ s._) e._ = ...;   /* 2 */
4     e._ s._ (t._) = ...;           /* 3 */
5     e._ (e._) (t._) = ...;         /* 4 */
6     (t._ t._) t._ (t._) = ...;     /* 5 */
7 }
```

1.4 Регулярное представление шаблонов

Регулярное представление шаблона представляет особый интерес, поскольку, в сравнении с контекстно-свободными языками (в частности, языками с мага-

зинных автоматов, управляемых входом), коими являются $\mathcal{W}_{\text{expr}}$ и $\mathcal{W}_{\text{pat}}$, регулярные языки более просты для анализа.

1.4.1 Каскадное произведение автоматов

Рассмотрим основную теоретическую конструкцию, позволяющую извлекать регулярное представление шаблонов — каскадное произведение автоматов:

Определение 15 (Каскадное произведение автоматов). Пусть $A_1 = \langle Q_1, \Sigma, \delta_1, q_{0_1}, F_1 \rangle$ и $A_2 = \langle Q_2, \Sigma \times Q_1, \delta_2, q_{0_2}, F_2 \rangle$ — детерминированные конечные автоматы. Каскадным произведением $A_1 \circ A_2$ называется ДКА $\langle Q, \Sigma, \delta, q_0, F \rangle$, где

$$\begin{aligned} Q &= Q_1 \times Q_2, \\ q_0 &= (q_{0_1}, q_{0_2}), \\ F &= F_1 \times F_2, \\ \delta((q_1, q_2), a) &= (\delta_1(q_1, a), \delta_2(q_2, (a, q_1))). \end{aligned}$$

Иногда будем просто говорить о каскаде (двух и более автоматов), подразумевая каскадное произведение автоматов.

Определение, данное выше, является вариантом прямого произведения автоматов. Как и многие другие операции над ДКА, использующие прямое произведение (например, пересечение и объединение), каскадное произведение можно описать как систему из двух автоматов, работающих параллельно на одном и том же входном слове. В случае каскада двух автоматов между ними проявляется односторонняя зависимость: оба автомата синхронно читают посимвольно входное слово $w \in \Sigma^*$, при этом первый автомат A работает автономно над алфавитом Σ , а второй автомат, помимо символа $w[i]$, использует текущее (до перехода по $w[i]$) состояние первого автомата q , т.е. работает над алфавитом $\Sigma \times Q_1$.

Основная идея заключается в делегировании распознавания вложенных подшаблонов элементам каскада.

Введем искомый алфавит $\Gamma_0 = \{b, s, (,)\}$. b является мета-символом, соответствующим словам $\mathcal{W}_{\text{expr}} \ni w = (v)$, $v \in \mathcal{W}_{\text{expr}}$, s также является мета-символом, соответствующим словам $\mathcal{W}_{\text{expr}} \ni w = \gamma \in \Sigma_{\text{int}}$. Будем рассматривать

регулярные выражения r над алфавитом Γ_0 и ДКА A над этим же алфавитом. Будем обозначать множество слов $\{w \mid w \in \Gamma_0^*\}$, принадлежащие языку r (A), как $R(r)$ ($R(A)$). Отклонение от стандартного обозначения через $\mathcal{L}$ сделано во избежание путаницы: каждое слово языка r (A) в силу смысла символов $\gamma \in \Gamma_0$ само задает некоторый язык слов-выражений из $\mathcal{W}_{\text{expr}}$. Для слова $u \in R(r)$ поставим ему в соответствие слово $u_{\mathbb{N}}$ над $\Gamma_{\mathbb{N}} = \{ (,) \} \cup \{b, s\} \times \mathbb{N}$ с пронумерованными переменными, аналогично тому, как это было сделано для шаблонов $p \in \mathcal{W}_{\text{pat}}$ и $p_{\mathbb{N}}$. Будем говорить, что w допускает сопоставление с r , если

$$\exists u \in R(r) \exists \phi: \{b, s\} \times \mathbb{N} \rightarrow \mathcal{W}_{\text{expr}} \mid \phi(u_{\mathbb{N}}) = w,$$

при этом

$$\begin{aligned} \phi((b, i)) &\mapsto (w) \in \mathcal{W}_{\text{expr}} \forall i \in \mathbb{N}; \\ \phi((s, i)) &\mapsto \gamma \in \Sigma_{\text{int}} \forall i \in \mathbb{N}; \end{aligned}$$

Множество слов $w \in \mathcal{W}_{\text{expr}}$, допускающих сопоставление с r , будем обозначать как $\mathcal{L}(r)$. Любой шаблон $p \in \mathcal{W}_{\text{pat}}$ можно представить в виде регулярного выражения r над алфавитом Γ так, что $\mathcal{L}(p) = \mathcal{L}(r)$. Данный факт следует непосредственно из определения самих e, t, s -символов, ниже представлено соответствие между символами и регулярными выражениями:

- e -символ соответствует выражению $(b|s)^*$;
- t -символ соответствует выражению $(b|s)$;
- s -символ соответствует выражению s .

Введем функцию глубины вложенности на элементах языка правильно вложенных слов:

$$d(w, i) = \begin{cases} 0, & i = 0 \\ d(w, i - 1) + 1, & w[i] \in \Sigma_{\text{call}} \\ d(w, i - 1) - 1, & w[i] \in \Sigma_{\text{ret}} \\ d(w, i - 1), & w[i] \in \Sigma_{\text{int}} \end{cases}$$

Тогда максимальным уровнем вложенности некоторого слова w будет

$$\max d(w) = \max_{i \in \overline{1, |w|}} d(w[i]).$$

Подшаблоном p уровня вложенности k (или просто k -подшаблоном) будем называть подслово $w[i : j]$ такое, что

$$d(w, i) = k \wedge d(w, j) = k - 1 \wedge \forall m \in \overline{i + 1, j - 1} \ d(w, m) \geq k - 1.$$

Обозначим множество всех k -подшаблонов p как **subpat** _{k} (p).

Рассмотрим теперь теоретическую конструкцию каскада по данному шаблону p . Сначала для $k = \max(p)$ построим регулярные выражения r_i над алфавитом Γ_0 по правилам, описанным выше, для всех k -подшаблонов p_i ; далее построим соответствующие ДКА A_i .

Рассмотрим ДКА-объединение A всех A_i с размеченными состояниями: каждое финальное состояние f автомата A помечается множеством $\alpha \subset \mathbf{subpat}_k(p)$, если все пути в автомате A до состояния f принадлежат языкам регулярных выражений, соответствующим k -подшаблонам $p_j \in \alpha$. Чтобы автомат корректно работал только с k -шаблонами, строится композиция автомата A с автоматом-лифтом на уровень скобочной вложенности k . Начальным состоянием A становится начальное состояние лифта, последний «этаж» лифта совмещается с бывшим начальным состоянием A . Пример автомата-лифта для уровня 2 приведен на рисунке РИС!.

Полученный автомат A является управляющим автоматом каскада. Если $k > 1$, то следующий элемент каскада будет управляться символом входного слова и состоянием A . Для построения управляемого автомата B необходимо рассмотреть множество **subpat** _{$k-1$} , его элементы включают себя k -шаблоны в качестве подшаблонов. Для принятия решения о переходе по этим k -шаблонам автомат B использует состояния автомата A , аннотации финальных состояний A позволяют автомату B получать необходимую информацию о том, какие шаблоны прочитаны A .

Итак, конструкция выше задает пару автоматов, каскадное произведение которых $A \circ B$ распознает $k - 1$ -шаблоны. Повторяя процедуру выше для $k, k - 1, \dots, 0$ -шаблонов, получаем каскад $A_k \circ A_{k-1} \circ \dots \circ A_0$, распознающий искомый шаблон.

Рассмотрим пример функции, приведённой в листинге 3.

Листинг 3 — Пример функции на базисном Рефале

```
1 f {  
2   t.first (e.x (s.y) s.z) = ...;  
3   ((t.x) e.y s.z) s.last = ...;  
4 }
```

Для примера в листинге 3 шаблонами уровня вложенности 2 будут s, t ; уровня 1 — $e(s)s, (t)es$; без скобочной вложенности — $t(e(s)s), ((t)es)s$.

1.4.2 Переход от каскада к ДКА

Конструкция выше является теоретическим обоснованием подхода, описанного ниже. Покажем, что от явного построения каскада можно избавиться вовсе и обойтись лишь построением аннотированного ДКА-объединения для подшаблонов всех уровней вложенности.

Рассмотрим структуру любого управляемого элемента каскада. В любом состоянии после перехода по открывающей скобке управляемый автомат "ожидает" закрывающую скобку нужного уровня, фактически игнорируя все символы, поступающие на вход до нее. Этот факт потребуется в дальнейшем.

Введем на множестве состояний Q аннотированного автомата отношение эквивалентности $\sim_\alpha$, для $q_1, q_2 \in Q$ имеем $q_1 \sim_\alpha q_2$ тогда, и только тогда, когда аннотация q_1 совпадает с q_2 . Будем называть элементы этого фактора $b_0, b_1, \dots, b_n$.

Заметим, что все состояния элемента каскада с одинаковыми аннотациями неразличимы с точки зрения элемента младшего уровня, а также то, что состояния с уникальной аннотацией дизъюнкты относительно других языков путей к другим аннотациям. Выражаясь немного неформально, язык b_i не пересекается с языками $b_j, j \neq i$.

Воспользовавшись наблюдениями выше, можно свести каскад к построению отдельных ДКА в различных алфавитах по следующим правилам:

1. Первый управляющий автомат $A_k = \langle Q, q_0, F, \delta \rangle$ над $\{s, b\}$ строится как и ранее.
2. Строится фактор-множество $Q / \sim_\alpha$, при этом к нему добавляется элемент $\emptyset$, если в A_k есть хоть одно непринимаящее полезное состояние.

3. Младший элемент каскада строится над алфавитом $s, b_0, b_1, \dots, b_n$ следующим образом — по шаблону все так же можно построить регулярное выражение, но символ b теперь заменяется набором $b_0, b_1, \dots, b_n$. По регулярному выражению построить ДКА.
4. Если еще не построен ДКА для 0-шаблонов, перейти к шагу 2, иначе завершить построение.

Таким образом, в данной конструкции бремя различия несовпадающих вложенных подшаблонов ложится на алфавит. Конечным результатом алгоритма, следующего правилам выше, будет ДКА искомого шаблона в некотором алфавите $s, b_0, b_1, \dots, b_n$.

Заметим, что по построению общее число аннотаций на каждого уровне относительно числа k -шаблонов n есть $O(2^n)$. Именно этим обусловлен отказ от рассмотрения шаблонов, содержащих атомарные символы — в шаблонах с высоким уровнем вложенности различие констант может привести к очень сильному разрастанию алфавита.

1.5 Сужение шаблона как регулярный язык

Модифицировав немного конструкцию ДКА из раздела выше для построения автоматов по набору шаблонов в функции, и используя базовые операций над этими ДКА, можно построить сужения каждого шаблона.

При решении задачи экранирования необходимо различать множество $\bigcup_i \mathbf{subpat}_k(p_i)$ всех k -подшаблонов всех шаблонов некоторой функции f . Конструкция выше тривиально обобщается на несколько шаблонов: для всех k вместо $\mathbf{subpat}_k(p)$ конкретного шаблона рассматривается объединенное множество $\bigcup_i \mathbf{subpat}_k(p_i)$.

Упрощенная задача экранирования для функции f теперь тривиально решается проверкой $\mathcal{L}_{\text{fact}}(f.p_i) = \emptyset$ для всех $p_i \in \mathbf{pat} f$.

Для полного решения задачи экранирования для функции f , решим сначала упрощенный вариант задачи, а потом для каждого экранируемого шаблона p_i рассмотрим копию функции f , из которой поочередно будем пытаться удалять шаблоны $p_{i-1}, p_{i-2}, \dots, p_1$. Если при удалении очередного шаблона $f.p_j$ и последующем решении упрощенной задачи экранирования для оставшейся части функции

f результат будет пустым множеством, то шаблон p_j участвует в экранировании шаблона p_i и остается в функции. В противном случае он удаляется.

Теперь же рассмотрим сужения в контексте извлечения документации о шаблоне.

Рассмотрим пример гипотетической функции, приведённой в листинге 4.

Листинг 4 — Некоторая функция, обрабатывающая какие-то данные (в диалекте Рефал5-λ)

```

1 AnalyzeData {
2   t.node /* empty */ = <AnalyzeData <AppendData t.node>>;
3   t.node t.data, t.data : {
4     ...
5   };
6   e.otherwise = <Error>;
7 }
```

В данном примере очевидно, что шаблон третьего предложения *AnalyzeData.p₃* можно уточнить:

$$e.otherwise \rightsquigarrow t.1 \ t.2 \ t.3 \ e.4;$$

т.е. сужение шаблона *e.otherwise* совпадает с $\mathcal{L}(t.1 \ t.2 \ t.3)$.

Менее очевидный пример представлен в листинге 5. Для последнего шаблона можно вывести, что все слова, сопоставляющиеся с этим шаблоном, или начинаются с пустых скобок $()$, либо со слов, удовлетворяющих шаблону $((t.1 \ e.2) \ e.3)$.

Листинг 5 — Функция *h*

```

1 h {
2   () t.1 = ...;
3   (e.1 () e.3) e.4 = ...;
4   (s.1 e.1) e.2 = ...;
5   (e.1) e.2 s.1 e.3 = ...;
6   (e.1) e.2 = ...;
7 }
```

Обходя в ширину ДКА фактического языка 0-шаблонов можно строить дерево обязательных префиксов. Помимо этого, можно извлекать обязательные инфиксы. Это простое наблюдение нуждается в аккуратной реализации: в самом наглядном представлении обязательных префиксов — перечислении — может оказаться слишком много элементов. Из-за сложности нахождения всех обязательных инфиксов для данного ДКА, разумно использовать аппроксимацию — ДКА можно представить как граф, вершины которого — состояния ДКА, а ребра — переходы между состояниями (переходы между одинаковыми состояниями по разным символам схлопываются в одно ребро, которое помечается всеми этими символами). С помощью классического алгоритма Тарьяна в графе можно найти мосты. Дополнительно проверив, что ни одно финальное состояние не достигается без посещения «начала» моста, можно получить некоторые обязательные инфиксы языка ДКА.

2 Конструкторская часть

2.1 Общая архитектура программного комплекса

Было решено работать с диалектом Рефал5-λ, поскольку этот диалект, во-первых, является точным надмножеством Рефала-5, а во-вторых является современной реализацией Рефала, знакомой автору.

Рационально поделить программный комплекс на две части — библиотеку, содержащую всю логику работы с шаблонами, автоматами, алгоритмами и т.п., и приложение с интерфейсом командной строки, использующую эту библиотеку.

Устройство самой библиотеки следует классической архитектуре большинства фронтендов компиляторов. Таким образом, библиотека делится на три модуля:

- модуль `lexer` — модуль, отвечающий за лексический анализ;
- модуль `parser` — модуль, отвечающий за синтаксический анализ;
- модуль `sema` — модуль, отвечающий за семантический анализ.

Модуль `sema` единственный предоставляет публичные функции, доступные для вызова извне. Сами модули взаимодействуют друг с другом весьма ограничено, сохраняя свою внутреннюю логику закрытой. Схема взаимодействия модулей представлена на рисунке РИС!.

Лексический и синтаксический анализаторы не представляют особого интереса — Рефал5-λ имеет сравнительно простую лексическую структуру, а грамматика языка ССЫЛКА! допускает реализацию синтаксического анализа рекурсивным спуском. Так как основная цель работы — написание семантического анализатора, лексический и синтаксический анализаторы реализованы довольно просто, и не имеют продвинутых способов восстановления после ошибок.

2.2 Архитектура подсистемы семантического анализатора

2.2.1 Вспомогательные подмодули семантического анализатора

Семантический анализатор, будучи самым крупным модулем библиотеки, содержит в себе несколько подмодулей, выполняющих различные задачи.

Первым таким подмодулем является модуль `automata`, отвечающий за работу с автоматами и за способ хранения ДКА и НКА в памяти. При разработке прототипа приложения и его тестировании было установлено, что в абсолютном большинстве случаев автоматы имеют размер до десяти тысяч состояний, а размер алфавита не превышает одной тысячи. В связи с этим в итоге было выбрано сжатое разрежённое строковое представление для исходящих и входящих ребер. Пример такого представления приведен на рисунке РИС!. Остслеживать входящие ребра легко при построении ДКА, в дальнейшем это дает выигрыш при реализации алгоритмов минимизации и удаления бесполезных состояний. В числе операций на ДКА, реализованных в модуле, присутствуют:

- объединение;
- аннотированное объединение;
- пересечение;
- конкатенация (через прямое произведение);
- конкатенация (через построение НКА);
- дополнение;
- проверка языков автоматов на включение;
- минимизация;
- удаление бесполезных состояний.

Необходимость двух вариантов конкатенации ДКА будет обусловлена в следующем разделе. НКА в модуле нужны лишь для того, чтобы стать результатом конкатенации ДКА и сразу же быть детерминизированными.

Второй подмодуль `abstract_patterns` позволяет из конкретных узлов синтаксического дерева строить абстрактные шаблоны, эквивалентные шаблонам из $\mathcal{W}_{\text{pat}}$. Наличие такого модуля значительно облегчает дальнейшую обработку шаблонов. Ключевой функцией в модуле является функция получения из узла син-

таксического дерева определения функции набора абстрактных шаблонов и сопутствующей информации; наличие этой информации обусловлено необходимостью различать шаблоны, изначально удовлетворяющие определению шаблонов из $\mathcal{W}_{\text{pat}}$, от остальных. Модуль в том числе несет ответственность за преобразование всех шаблонов к виду, описанному в исследовательской части ССЫЛКА!. Информация об абстрактном шаблоне включает в себя следующие факты:

- были ли в искомом шаблоне повторные переменные;
- были ли в искомом шаблоне атомарные символы;
- были ли в предложении функции, соответствующем искомому шаблону, условия.

2.2.2 Проектирование алгоритмов

Центральным алгоритмом является построение ДКА абстрактного шаблона. Псевдокод этого алгоритма приведен в листинге ЛИСТИНГ!.

Другим важным алгоритмом, эксплуатирующим предыдущий, является построение сужения шаблона (см. ЛИСТИНГ!).

Имея ДКА абстрактного шаблона, можно использовать их для решения задачи распознавания перекрытий и получения документации.

Отметим, что сужать фактический язык шаблона могут лишь те шаблоны, язык которых «не искажен» различным образом. Поэтому в алгоритме участвуют в сопоставлении фактического языка только те шаблоны, которые изначально удовлетворяли определению шаблонов языка $\mathcal{W}_{\text{pat}}$ (абстрактизация конкретного шаблона всегда расширяет его язык). Еще одно требование — в предложениях шаблонов, участвующих в сужении, не должно быть условий (условия обычно искажают язык шаблона p в том смысле, что множество распознаваемых им слов из-за условия является лишь подмножеством $\mathcal{L}(p)$). Именно по этой причине необходимо сохранять информацию об условиях в абстрактных шаблонах.

Алгоритм решения упрощенной задачи распознавания перекрытий представлен в листинге ЛИСТИНГ!.

Алгоритм решения полной задачи распознавания перекрытий представлен в листинге ЛИСТИНГ!.

Обратим внимание, что при работе с реальными программами целесообразно сначала находить решение упрощенной задачи, так как экранирование не

является часто встречающимся явлением, а потом уже для функций, в которых встречается экранирование, находить решение полной задачи. Обусловлено это сложностью построения ДКА (в том числе промежуточных) с большим алфавитом (размер которого может в худшем случае быть экспоненциальным от количества различных шаблонов данного уровня вложенности). Операции над ДКА, необходимые для построения сужений, выполняются гораздо быстрее, но, тем не менее, если для решения классической задачи требуется порядка $O(N)$ операций для некоторого N , то для решения полной задачи сужения требуется уже порядка $O(N^2)$ операций.

Если сужение шаблона непусто, можно извлекать некоторые факты о нем. Одним из вариантов является анализ множества обязательных префиксов и обязательных инфиксов.

Дерево обязательных префиксов, построенное для ДКА шаблона 0-го уровня вложенности, может содержать символы $b_1, b_2, \dots, b_n$. Для человека такие символы никакой смысловой нагрузки почти не несут, поэтому для каждого символа строится свое дерево обязательных префиксов. Это продолжается до тех пор, пока не будет получено дерево над алфавитом $\{s, b\}$. Однако в результате склейки таких деревьев результирующее дерево может оказаться слишком большим. Для баланса между полнотой документации и ее размера используется эвристика, ограничивающая деревья на длину веток и число веток в целом. Причем чем выше уровень вложенности символа b_i , тем сильнее ограничивается число веток, а на длину ограничение небольшое. Чем ниже уровень вложенности символа b_i , тем сильнее ограничивается длина веток, на число веток накладывается все меньшее ограничение.

Алгоритм извлечения обязательных инфиксов работает с ДКА как с графом, при этом кратные ребра схлопываются в одно.

Два алгоритма извлечения документации к шаблонам функции представлены в листингах ЛИСТИНГ! и ЛИСТИНГ!.

2.3 Пользовательский интерфейс

Пользовательский интерфейс обязан предоставлять пользователю возможность воспользоваться основными функциями модуля семантического анализато-

ра, а именно решениями задачи распознавания перекрытий и извлечения документации к шаблонам функции.

Как уже было сказано, библиотеке сопутствует приложение с интерфейсом командной строки, использующее эту библиотеку. Это приложение позволяет пользователю указать файл с исходным кодом и выполнить одно из трех основных действий:

- решение упрощенной задачи распознавания перекрытий;
- решение полной задачи распознавания перекрытий;
- извлечение документации к шаблонам функции;

Для простоты взаимодействия с приложением весь вывод направляется в стандартный поток вывода в удобном для чтения формате. Таким образом, пользовательский интерфейс представляет собой интерфейс командной строки, позволяющий пользователю взаимодействовать с библиотекой с помощью указания флагов при запуске программы.

3 Технологическая часть

3.1 Обзор используемых технологий

3.2 Реализация модуля для работы с абстрактными шаблонами

3.3 Реализация модуля для работы с автоматами

3.4 Реализация алгоритма обнаружения перекрытий

3.5 Реализация алгоритма извлечения документации

3.6 Руководство пользователя

4 Аналитическая часть

ЗАКЛЮЧЕНИЕ

СПИСОК ЛИТЕРАТУРЫ